

Fundamental

代数基本定理

定义

任何复系数一元 n 次多项式（至少为 1 ）方程在复数域上至少有一根。

由此推出， n 次复系数多项式方程在复数域内有且只有 n 个根，重根按重数计算。

有时这个定理也表述为：

任何一个非零的一元 n 次复系数多项式，都正好有 n 个复数根。

代数基本定理的证明，一般会用到复变函数或者近世代数，因此往往作为一个熟知结论直接应用。

根据代数基本定理，一个复系数多项式一定可以唯一地分解为：

其中各个根均为复数， α_i 。

虚根成对定理

代数基本定理的研究对象是复系数多项式。当对实系数多项式进行研究时，虽然也能分解出复数根，却需要将研究范围扩大，不太方便。

虚根：非实数根。

定理：实系数多项式的根的共轭复数也是该多项式的根。

证明：直接在代数基本定理的等式两端取共轭即证毕。

如果根本身是实数，则取共轭仍为它本身，不受影响。

如果根是虚根，则虚根的共轭复数也是原多项式的根。那么，两个虚根就可以配对。

定理：实数系数方程的共轭虚根一定成对出现，并且共轭虚根的重数相等。

证明：假设一个根为 α ，则另一个根为 $\bar{\alpha}$ 。这意味着在分解式中存在两项：

可以看到两项乘在一起，各项系数会全部变为实数。这个等式右端的二次实系数多项式整除原始的多项式。

于是，在代数基本定理的等式中，两遍同时除以这个二次三项式，得到的仍旧是实系数多项式的等式。对新等式重复操作，随着次数的下降，若干次后即不存在虚根。

因此，每对共轭虚根的重数相等。证毕。

以下是虚根成对定理的推论：

- 实系数奇次多项式至少有一个实根，并且总共有奇数个实根。
- 实系数偶次多项式可能没有实根，总共有偶数个实根。

称上述二次三项式为二次实系数不可约因式。不可约是指它在实数范围内不可约。

定理：实系数多项式一定是一次或者二次实系数不可约因式的积。

证明：

只要实系数多项式有一个实根，就有一个实系数因式 和它对应；有一对虚根，就有一个实系数因式 和它对应。

因此，只要在原始的代数基本定理分解式中，利用虚根成对定理进行配对，即证毕。

根据虚根成对定理，一个实系数多项式 一定可以唯一地分解为：

其中各项系数均为实数，。

林士谔算法

简介

怎样对实系数多项式进行代数基本定理的分解？如果将数域扩充至复数会很复杂。

如果只在实数范围内进行分解，只能保证，当次数大于 的时候，一定存在实系数二次三项式因式。

这是因为，如果该多项式有虚根，直接凑出一对共轭虚根即可。如果该多项式只有实根，任取两个实根对应的一次因式乘在一起，也能得到实系数二次三项式因式。

找到二次三项式因式之后，再从二次式中解实根或复根就极为容易。于是便有逐次 找出一个二次因子 来求得方程的复根的计算方法，这种方法避免了复数运算。

在 1940 年 8 月、1943 年 8 月和 1947 年 7 月，林士谔先后在 MIT 出版的《数学物理》杂志上接连正式发表了 3 篇关于解算高阶方程式复根方法的论文¹，每次均有改进。

这个方法今天还在现代计算机中进行快速运算，计算机程序包（如 MATLAB）中的多项式求根程序依据的原理也是这个算法。

过程

要想找到一个二次三项式因子，就要将多项式分解为：

由于无法一下子找到二次三项式因子，按照迭代求解的思路，对于初始值有：

会产生一个一次式作为余项。只要余项足够小，即可近似地找到待求因子。

我们希望最终解是初始值加一个偏移修正：

余式中的两个数 由除式的给定系数 决定。有偏导数关系：

在初始的等式中，被除式 是给定的，商式 和余式 随着除式 的变化而变化。因此有偏导数关系

注意到，偏导数只是一个数值，与变元 无关。因此有整除关系

这里的结论是，待求的偏导数，恰好是对商式继续做除法的余式。多项式对给定二次三项式的除法，直接计算即可。这里就求得了四个偏导数。

我们希望 p 和 q 加上偏移 dp 与 dq 得到 p' 与 q' ，即 p' 与 q' 是 p 和 q 的相反数。因此要解方程：

从上述方程组中解得 dp 和 dq 相应的偏移 dp 和 dq ，直接用二阶行列式求解即可。

实现

```
1 // a 是原始的多项式, n 是多项式次数, p 是待求的一次项, q 是待求的常数项
2 void Shie(double a[], int n, double *p, double *q) {
3     // 数组 b 是多项式 a 除以当前迭代二次三项式的商
4     memset(b, 0, sizeof(b));
5     // 数组 c 是多项式 b 乘以 x 平方再除以当前迭代二次三项式的商
6     memset(c, 0, sizeof(c));
7     *p = 0;
8     *q = 0;
9     double dp = 1;
10    double dq = 1;
11    while (dp > eps || dp < -eps || dq > eps || dq < -eps) // eps 自行设定
12    {
13        double p0 = p;
14        double q0 = q;
15        b[n - 2] = a[n];
16        c[n - 2] = b[n - 2];
17        b[n - 3] = a[n - 1] - p0 * b[n - 2];
18        c[n - 3] = b[n - 3] - p0 * b[n - 2];
19        int j;
20        for (j = n - 4; j >= 0; j--) {
21            b[j] = a[j + 2] - p0 * b[j + 1] - q0 * b[j + 2];
22            c[j] = b[j] - p0 * c[j + 1] - q0 * c[j + 2];
23        }
24        double r = a[1] - p0 * b[0] - q0 * b[1];
25        double s = a[0] - q0 * b[0];
26        double rp = c[1];
27        double sp = b[0] - q0 * c[2];
28        double rq = c[0];
29        double sq = -q0 * c[1];
30        dp = (rp * s - r * sp) / (rp * sq - rq * sp);
31        dq = (r * sq - rq * s) / (rp * sq - rq * sp);
32        *p += dp;
33        *q += dq;
34    }
35 }
```

参考资料与注释

-
1. [林士谔. 论劈因法解高阶特征方程根值的应用问题. 数学进展, 1963\(03\):207-217. ↗](#)
-

本页面最近更新：2023/7/30 10:47:50, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者: [Enter-tainer](#), [Great-designer](#), [iamtwz](#), [Jijidawang](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用